Drone Delivery System Automation

INF212 Project 3

Prepared by
ID 240102002088
Hamza Enes Balahoroğlu

INF 212 Algorithms and Programming II
Spring 2026
Department of Electronic Engineering

Date Submitted: 30.04.2026

Project Objective

This project is a terminal-based system written in C. The main purpose is to manage customers, products, and warehouses using dynamic memory structures. Another important goal is to determine the warehouse closest to a customer during an order and calculate the total cost accordingly.

Description of Problem

In this project, the main problem is to improve the efficiency of a delivery system. When a customer places an order, the system should find the nearest warehouse and calculate the shipping cost in a simple and effective way.s

To achieve this, customer and warehouse locations are defined on a coordinate plane. The system must also track product stock and handle memory usage correctly to avoid issues such as memory leaks.

Description of Method

The primary goal of the system is to determine the warehouse nearest to the customer and calculate the associated logistics cost. To achieve this, Linear Search and Euclidean Distance methods are utilized together.In the first stage, the customer ID and product ID entered by the user are validated. This process is performed by conducting a linear search through the relevant linked lists and has a time complexity of O(N). During this step, both the customer's registration in the system and the stock status of the requested product are checked.Following the validation process, a scan is performed across all warehouses to find the one nearest to the customer. For each warehouse, the distance between the customer and the warehouse is calculated using the Euclidean distance formula:

$$d = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2}$$

Each calculated distance value is compared against the minimum distance found up to that point. When a smaller distance is detected, both the minimum distance value and the reference representing the nearest warehouse are updated. This process continues until all warehouses have been examined.

In the final stage, the logistics cost is calculated based on the shortest distance found. The total cost is obtained by multiplying the distance value by a fixed unit shipping fee (5 TL). Upon completion of the purchase, the stock quantity of the relevant product is updated and decreased.

User’s guide

The program features an interactive, menu-based structure where operations are performed by selecting numerical options from 1 to 8 via the main menu. To define customers and warehouses, the system collects ID information along with X-Y coordinates; for products, it requires the ID, name, price, and stock quantity. During a purchase transaction, the user provides the specific customer ID, product ID, and the requested quantity (units).

The program outputs data in the form of organized lists displayed on the screen. Notably, when an order analysis (Menu Option 7) is performed, the system presents a detailed "Invoice Summary" including the nearest warehouse ID, the distance between the customer and the warehouse, the calculated shipping fee, product cost, and the updated stock status following the transaction.

Certain error conditions may arise based on user inputs. Text inputs are handled via the DSTR_GetLine function, which returns data of the DString (dynamic string) type. This function is structured to allocate memory based on the size of the entered text, thereby eliminating limitations regarding the length of string inputs. In cases where incorrect data (such as letters) is entered into fields expecting numerical values, specialized Get_Int and Get_Float functions prevent program crashes and prompt the user for correct data entry.

If a customer or product ID that is not registered in the system is entered, a "Customer/Product not found" warning is issued. Furthermore, if the requested quantity exceeds the available stock, the transaction is canceled and an "Insufficient Stock" warning is displayed to the user.

Results of the solution

Adding a Customer, Product, and Warehouses:

========== ANAMENU ==========
1. Musteri Ekle
...
Seciminiz: 1
Musteri ID giriniz: 101
Musteri Ismi giriniz: Hamza
X Koordinati: 3.4
Y Koordinati: 1.2
Musteri basariyla eklendi!

Seciminiz: 2
Urun ID giriniz: 201
Urun Adi giriniz: Ekmek
Birim Fiyat (TL): 10
Stok Miktari: 20
Urun basariyla eklendi!

Seciminiz: 3
Depo ID giriniz: 301
Depo X Koordinati: 5
Depo Y Koordinati: 6.11
Depo basariyla eklendi!

Seciminiz: 3
Depo ID giriniz: 1
Depo X Koordinati: 1
Depo Y Koordinati: 3
Depo basariyla eklendi!

System Records and List Verification (Output):
--- MUSTERI LISTESI ---
ID : 101 Adi: Hamza x = 3.00 y = 1.00

--- URUN LISTESI ---
ID : 201 Urun: Ekmek Fiyat: 10.00 TL Stok: 20

--- DEPO LISTESI ---
ID : 1 x = 1.00 y = 3.00
ID : 301 x = 5.00 y = 6.00

Purchase Analysis & Cost Calculation:
========== ANAMENU ==========

```
Seciminiz: 7

--- SATIN ALMA ISLEMI ---

--- MUSTERI LISTESI ---
ID : 101            Adi: Hamza          x = 3.00  y = 1.00
Musteri ID giriniz: 101

--- URUN LISTESI ---
ID : 201            Urun: Ekmek     Fiyat: 10.00 TL Stok: 20
Satin alinacak Urun ID giriniz: 201
Kac adet alinacak?: 5

================ FATURA VE ANALIZ ================
Satin Alan Musteri ID : 101
Alinan Urun ID       : 201 (Miktar: 5 adet)
Gonderici Depo ID    : 1
Mesafe               : 2.83 km
--------------------------------------------------
Urun Bedeli          : 50.00 TL
Kargo Bedeli         : 14.14 TL (2.83 x 5 TL)
TOPLAM TUTAR         : 64.14 TL
Kalan Urun Stoku     : 15
==================================================
```

Flowchart and Pseudocode of the Program

Pseudocode:

```
GLOBAL POINTERS (Heads of the Linked Lists):
   customer_list = NULL
   product_list = NULL
   storehouse_list = NULL


FUNCTION Main():
   LOOP (Infinite):
      Print Main Menu to Screen (Options 1 to 8)
      choice = Get Integer Input from User

      SWITCH choice:
         CASE 1: Call Handle_Add_Customer()
         CASE 2: Call Handle_Add_Product()
         CASE 3: Call Handle_Add_Storehouse()
         CASE 4: Call View_All_Customers()
         CASE 5: Call View_All_Products()
         CASE 6: Call View_All_Storehouses()
         CASE 7: Call Analyze_Purchase()
         CASE 8:
```

```
            Call Quit_System()
            Break Loop and Exit Program
        DEFAULT:
            Print "Invalid choice!" to Screen
    END SWITCH
  END LOOP
END FUNCTION



FUNCTION Handle_Add_Customer():
  Get id, name, X, and Y coordinates from User

  new_customer = Allocate Memory for Customer(id, name, X, Y)

  new_customer.next = customer_list

  customer_list = new_customer

  Print "Successfully added" to Screen
END FUNCTION

FUNCTION Handle_Add_Product():
  Get id, name, price, and quantity from User

  new_product = Allocate Memory for Product(id, name, price, quantity)

  new_product.next = product_list

  product_list = new_product

  Print "Successfully added" to Screen
END FUNCTION

FUNCTION Handle_Add_Storehouse():
  Get id, X, and Y coordinates from User

  new_store = Allocate Memory for Storehouse(id, X, Y)

  new_store.next = storehouse_list

  storehouse_list = new_store

  Print "Successfully added" to Screen
END FUNCTION


FUNCTION View_All_Customers():
  Print "--- CUSTOMERS ---" to Screen
```

```
    current_node = customer_list

    WHILE current_node is NOT NULL:
        Print current_node details
        current_node = current_node.next // Jump to the previously added element
    END WHILE
END FUNCTION

FUNCTION Analyze_Purchase():
    IF customer_list is NULL OR product_list is NULL OR storehouse_list is NULL:
        Print "Missing data!" to Screen and EXIT FUNCTION

    Call View_All_Customers()
    target_customer_id = Get Input from User
    selected_customer = Search for target_customer_id by traversing customer_list
    IF selected_customer NOT FOUND: Print Error and EXIT FUNCTION

    Call View_All_Products()
    target_product_id = Get Input from User
    selected_product = Search for target_product_id by traversing product_list
    IF selected_product NOT FOUND: Print Error and EXIT FUNCTION

    buy_quantity = Get Input from User
    IF selected_product.stock < buy_quantity:
        Print "Insufficient Stock" to Screen and EXIT FUNCTION

    min_distance = INFINITY
    nearest_store = NULL

    current_store = storehouse_list
    WHILE current_store is NOT NULL:
        distance = Calculate_Euclidean_Distance(selected_customer, current_store)

        IF distance < min_distance:
            min_distance = distance
            nearest_store = current_store

        current_store = current_store.next
    END WHILE

    product_cost = selected_product.price * buy_quantity
    shipping_cost = min_distance * 5.0
    total_cost = product_cost + shipping_cost
    selected_product.stock = selected_product.stock - buy_quantity

    Print Invoice Details and Total Cost to Screen
END FUNCTION
```

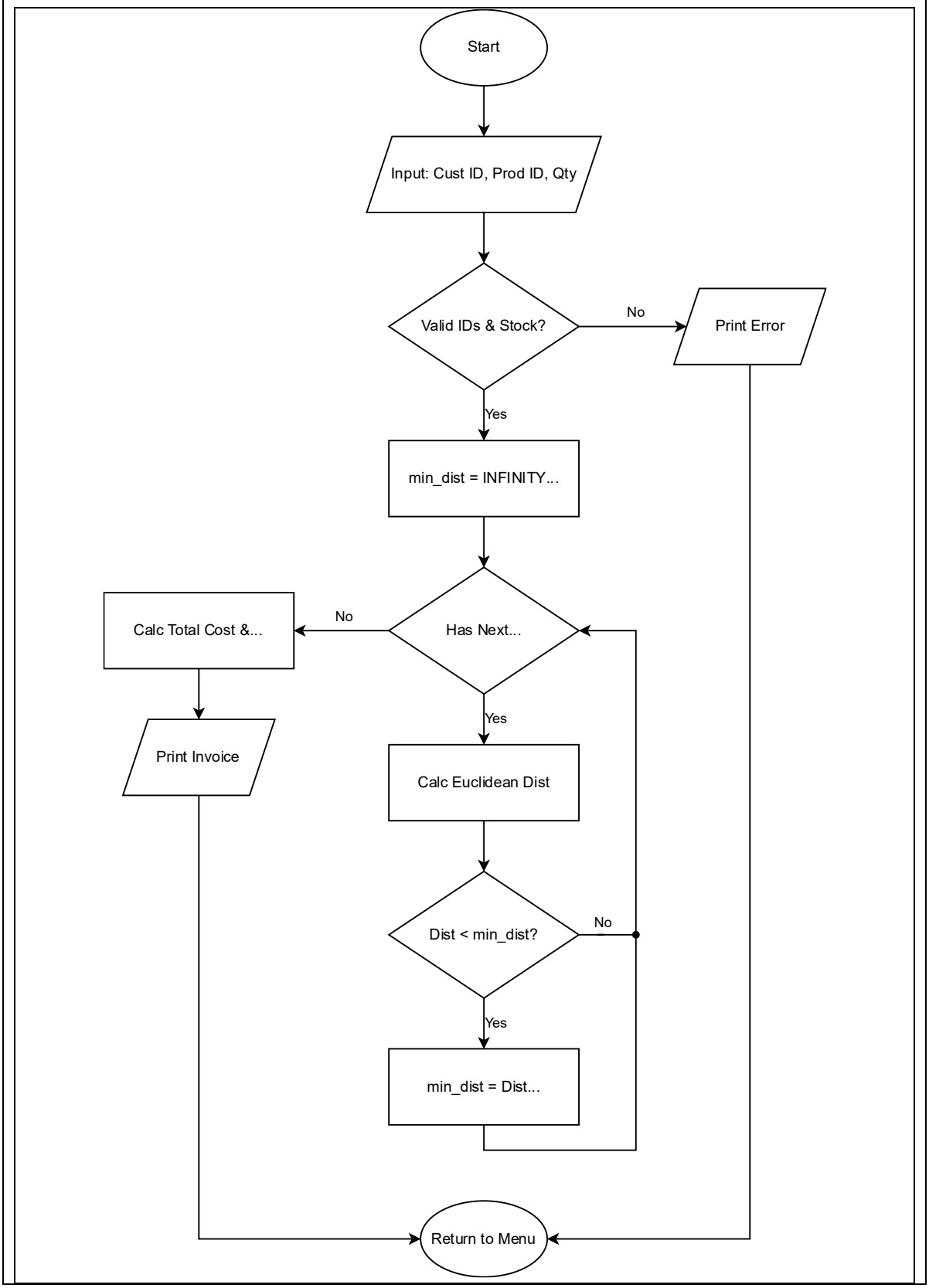
```
FUNCTION Quit_System():
   //  Clean up Customer List
   current_customer = customer_list
   WHILE current_customer is NOT NULL:
      next_customer = current_customer.next
      Call Object_Kill(current_customer)
      current_customer = next_customer
   END WHILE

   // Clean up Product List
   current_product = product_list
   WHILE current_product is NOT NULL:
      next_product = current_product.next
      Call Object_Kill(current_product)
      current_product = next_product
   END WHILE

   // Clean up Storehouse List
   current_store = storehouse_list
   WHILE current_store is NOT NULL:
      next_store = current_store.next
      Call Object_Kill(current_store)
      current_store = next_store
   END WHILE

   Print "Exiting system..." to Screen
END FUNCTION
```

The Flowchart For The Analyze_Purchase Algorithm:



Conclusion and Remarks

The developed program operates accurately and stably, fulfilling all expected functions, including the targeted "Purchase Analysis" and "Distance Calculation" algorithms. During testing, the system was observed to run without any errors. However, there are a few limitations based on the design choices.

The program is designed to run entirely on volatile dynamic memory (RAM). Since there is no File I/O mechanism or database integration, the data must be re-entered every time the program is executed. This creates a usability disadvantage, especially for larger datasets.

Additionally, while "add/create" operations on the linked lists are executed flawlessly, a specific mechanism to delete a node during runtime was not included in the project. Memory cleanup is only performed in bulk when the program is terminated.

This project allowed me to solidify my existing knowledge of C programming, particularly regarding memory management and pointer usage. One of the technical details I encountered during the development process involved self-referencing structs.

Due to the way the C compiler works, directly using a typedef is not sufficient if you need to use a pointer referencing the structure before its definition is complete. Therefore, I learned that a "struct tag" must be defined to use a notation like struct Customer *next within the structure. Applying this approach allowed me to resolve the "incompatible pointer type" error I initially faced.

Furthermore, the architecture I implemented provided a concrete opportunity to observe the deep structural similarities between Python and C. This relationship, stemming from Python's core being written in C (CPython), became especially apparent when simulating Object-Oriented Programming (OOP) concepts in C. For instance, the structure of the int STORE_Kill(void *self, void *arg) function I wrote—where the object takes its own reference as the first parameter via the self pointer—relies on the exact same underlying logic as defining a class method in Python, such as def storekill(self):. This experience gave me a much clearer understanding of how the concepts abstracted by high-level languages are actually constructed at a lower level using C.

Text of Program

// File : 240102002088_headers.h

```
#ifndef __PROGRAM_DATATYPES__
#define __PROGRAM_DATATYPES__
#include <stddef.h>

typedef int (*ActionFunc)(void *self, void *arg);

typedef struct
{
   ActionFunc present;
   ActionFunc kill;
} EntityHeader;

typedef struct
```

```
{
   const EntityHeader *base;
} GenericObject;

typedef struct
{
   const EntityHeader *base;
   float x;
   float y;
} Location;

typedef struct
{
   const EntityHeader *base;
   char *chr;
   size_t count;
} DString;

typedef struct Customer
{
   const EntityHeader *base;
   struct Customer *next;
   Location *location;
   DString *name; // Structure padding
   int id;
} Customer;

typedef struct Product
{
   const EntityHeader *base;
   struct Product *next;
   DString *name;
   float price;
   int quantity;
   int id;
} Product;

typedef struct Storehouse
{
   const EntityHeader *base;
   struct Storehouse *next;
   Location *location;
   int id;
} Storehouse;

#endif

#ifndef __PROGRAM_FUNCTIONS__
#define __PROGRAM_FUNCTIONS__
```

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

DString *DSTR_Create(char *char_list, int count);
DString *DSTR_GetLine();
int DSTR_Present(void *self, void *arg);
int DSTR_Kill(void *self, void *arg);

Location *LOC_Create(int x, int y);
float LOC_CalculateDistance(Location *l1, Location *l2);
int LOC_Present(void *self, void *arg);

Customer *CUS_Create(int id, DString *name, int x, int y);
int CUS_Present(void *self, void *arg);
int CUS_Kill(void *self, void *arg);

Storehouse *STORE_Create(int id, int x, int y);
int STORE_Kill(void *self, void *arg);
int STORE_Present(void *self, void *arg);

Product *PROD_Create(int id, DString *name, float price, int quantity);
int PROD_Kill(void *self, void *arg);
int PROD_Present(void *self, void *arg);

void Add_To_Customer_List(Customer *new_cus);
void Add_To_Product_List(Product *new_prod);
void Add_To_Store_List(Storehouse *new_store);

void Handle_Add_Customer(void);
void Handle_Add_Product(void);
void Handle_Add_Storehouse(void);

void View_All_Customers(void);
void View_All_Products(void);
void View_All_Storehouses(void);

void Quit_System();

void Analyze_Purchase();

int Get_Int(const char *prompt);
float Get_Float(const char *prompt);
int General_Free(void *self, void *arg);
int Object_Present(void *self);
int Object_Kill(void *self);

#endif
```

```
// File : 240102002088_main.c

#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include "240102002088_headers.h"

int main()
{
   int choice;
   while (1)
   {
      printf("\n========== ANAMENU ==========\n");
      printf("1. Musteri Ekle\n");
      printf("2. Urun Ekle\n");
      printf("3. Depo Ekle\n");
      printf("4. Musterileri Listele\n");
      printf("5. Urunleri Listele\n");
      printf("6. Depolari Listele\n");
      printf("7. Siparis Olustur\n");
      printf("8. Cikis\n");

      choice = Get_Int("Seciminiz: ");

      switch (choice)
      {
      case 1:
         Handle_Add_Customer();
         break;
      case 2:
         Handle_Add_Product();
         break;
      case 3:
         Handle_Add_Storehouse();
         break;
      case 4:
         View_All_Customers();
         break;
      case 5:
         View_All_Products();
         break;
      case 6:
         View_All_Storehouses();
         break;
      case 7:
         Analyze_Purchase();
         break;
```

```
        case 8:
            Quit_System();
            return 0;
        default:
            printf("Gecersiz secim!\n");
        }
    }
    return 0;
}
```

// File : 240102002088_funcions.c

```
#include "240102002088_headers.h"

//___________ Constant Variables ____________

const EntityHeader Customer_Functions = {
    .kill = CUS_Kill,
    .present = CUS_Present};

const EntityHeader Location_Functions = {
    .kill = General_Free,
    .present = LOC_Present};

const EntityHeader DString_Functions = {
    .kill = DSTR_Kill,
    .present = DSTR_Present};

const EntityHeader Store_Functions = {
    .kill = STORE_Kill,
    .present = STORE_Present};

const EntityHeader Product_Functions = {
    .kill = PROD_Kill,
    .present = PROD_Present};

//_________________ Variables _________________
Customer *customer_list = NULL;
Product *product_list = NULL;
Storehouse *storehouse_list = NULL;

Product *PROD_Create(int id, DString *name, float price, int quantity)
{
    Product *ptr = (Product *)malloc(sizeof(Product));
    if (ptr == NULL)
    {
        return NULL;
    }
```

```
    ptr->base = &Product_Functions;
    ptr->next = NULL;
    ptr->id = id;
    ptr->name = name;
    ptr->price = price;
    ptr->quantity = quantity;

    return ptr;
}

int PROD_Kill(void *self, void *arg)
{
    if (self == NULL)
        return 0;

    Product *prod = (Product *)self;

    if (prod->name != NULL && prod->name->base != NULL && prod->name->base->kill != NULL)
    {
        Object_Kill(prod->name);
    }

    free(prod);
    return 1;
}

int PROD_Present(void *self, void *arg)
{
    if (self == NULL)
        return 0;

    Product *prod = (Product *)self;

    printf("ID : %d\t\tUrun: ", prod->id);

    Object_Present(prod->name);

    printf("\tFiyat: %.2f TL\tStok: %d\n", prod->price, prod->quantity);

    return 1;
}

Storehouse *STORE_Create(int id, float x, float y)
{
    Storehouse *store = calloc(1, sizeof(Storehouse));

    if (store == NULL)
    {
        return NULL;
```

```
    }

    store->base = &Store_Functions;
    store->id = id;

    Location *temp_loc = LOC_Create(x, y);

    if (temp_loc == NULL)
    {
        Object_Kill(store);
        return NULL;
    }

    store->location = temp_loc;
    store->next = NULL;

    return store;
}

int STORE_Kill(void *self, void *arg)
{
    if (self == NULL)
        return 0;

    Storehouse *store = (Storehouse *)self;

    Object_Kill(store->location);

    free(store);
    return 1;
}

int STORE_Present(void *self, void *arg)
{
    if (self == NULL)
        return 0;

    Storehouse *store = (Storehouse *)self;

    printf("ID : %d\t\t", store->id);
    Object_Present(store->location);

    printf("\n");
    return 1;
}

Location *LOC_Create(float x, float y)
{
    Location *ptr = calloc(1, sizeof(Location));
```

```
    if (ptr == NULL)
    {
        return NULL;
    }

    ptr->base = &Location_Functions;
    ptr->x = x;
    ptr->y = y;

    return ptr;
}

Customer *CUS_Create(int id, DString *name, float x, float y)
{
    if (name == NULL)
        return NULL;

    Customer *customer = (Customer *)calloc(1, sizeof(Customer));

    if (customer == NULL)
        return NULL;

    customer->base = &Customer_Functions;
    customer->id = id;
    customer->name = name;

    Location *temp_loc = LOC_Create(x, y);

    if (temp_loc == NULL)
    {
        Object_Kill(customer);
        return NULL;
    }

    customer->location = temp_loc;

    customer->next = NULL;
    return customer;
}

int CUS_Kill(void *self, void *arg)
{
    Customer *cus = (Customer *)self;

    Object_Kill(cus->name);
    cus->name = NULL;

    Object_Kill(cus->location);
```

```
    cus->location = NULL;

    free(cus);

    return 1;
}

DString *DSTR_Create(char *char_list, int count)
{
    DString *str = (DString *)calloc(1, sizeof(DString));
    if (str == NULL)
        return NULL;

    char *chr = (char *)calloc(count + 1, sizeof(char));

    if (chr == NULL)
    {
        free(str);
        return NULL;
    }

    str->chr = chr;
    str->count = count;
    str->base = &DString_Functions;

    for (int i = 0; i < count; i++)
    {
        str->chr[i] = char_list[i];
    }

    str->chr[count] = '\0';

    return str;
}

int DSTR_Kill(void *self, void *arg)
{
    DString *str = (DString *)self;

    free(str->chr);
    str->chr = NULL;

    free(str);

    return 1;
}

int DSTR_Present(void *self, void *arg)
{
```

```
    if (self == NULL)
    {
        return 0;
    }

    DString *str = (DString *)self;

    if (str->chr == NULL)
    {
        return 0;
    }

    printf("%s", str->chr);

    return 1;
}

DString *DSTR_GetLine()
{
    int capacity = 8; // Başlangıç kapasitesi
    int length = 0;
    char *buffer = (char *)malloc(capacity * sizeof(char));
    int ch;

    if (buffer == NULL)
        return NULL;

    while ((ch = getchar()) != '\n' && ch != EOF)
    {

        if (length + 1 >= capacity)
        {
            capacity *= 2;
            char *temp = (char *)realloc(buffer, capacity * sizeof(char));
            if (temp == NULL)
            {
                free(buffer);
                return NULL;
            }
            buffer = temp;
        }
        buffer[length++] = (char)ch;
    }

    DString *str = DSTR_Create(buffer, length);

    free(buffer);

    return str;
```

```
}

int General_Free(void *self, void *arg)
{
   free(self);
   return 1;
}

int Object_Kill(void *self)
{
   if (self == NULL)
      return 0;

   GenericObject *obj = (GenericObject *)self;

   if (obj->base == NULL || obj->base->kill == NULL)
      return 0;

   obj->base->kill(obj, NULL);
   return 1;
}

int Object_Present(void *self)
{
   if (self == NULL)
      return 0;

   GenericObject *obj = (GenericObject *)self;

   if (obj->base == NULL || obj->base->present == NULL)
      return 0;

   obj->base->present(obj, NULL);
   return 1;
}

int LOC_Present(void *self, void *arg)
{
   Location *loc = (Location *)self;

   printf("x = %.2f  y = %.2f", loc->x, loc->y);
   return 1;
}

int CUS_Present(void *self, void *arg)
{
   Customer *cus = (Customer *)self;
   Location *loc = (Location *)cus->location;
   DString *name = (DString *)cus->name;
```

```
   printf("ID : %d\t\tAdi: ", cus->id);

   Object_Present(name);

   printf("\t\t");

   Object_Present(loc);

   printf("\n");

   return 1;
}

float LOC_CalculateDistance(Location *l1, Location *l2)
{
   if (l1 == NULL || l2 == NULL)
      return 0.0;

   float dx = l2->x - l1->x;
   float dy = l2->y - l1->y;
   return sqrt(dx * dx + dy * dy);
}

int Get_Int(const char *prompt)
{
   int value;
   int status;

   while (1)
   {
      printf("%s", prompt);
      status = scanf("%d", &value);

      if (status == 1)
      {
         while (getchar() != '\n')
            ;
         return value;
      }
      else
      {
         // Harf veya yanlis bir sey girildi.
         printf("Hatali giris! Lutfen sadece sayi girin.\n");
         while (getchar() != '\n')
            ;
      }
   }
}
```

```
float Get_Float(const char *prompt)
{
   float value;
   int status;

   while (1)
   {
      printf("%s", prompt);
      status = scanf("%f", &value);

      if (status == 1)
      {
         while (getchar() != '\n')
            ;
         return value;
      }
      else
      {
         printf("Hatali giris! Lutfen sadece sayi (orn: 1.5) girin.\n");
         while (getchar() != '\n')
            ;
      }
   }
}

void Add_To_Customer_List(Customer *new_cus)
{
   new_cus->next = customer_list;
   customer_list = new_cus;
}

void Add_To_Product_List(Product *new_prod)
{
   new_prod->next = product_list;
   product_list = new_prod;
}

void Add_To_Store_List(Storehouse *new_store)
{
   new_store->next = storehouse_list;
   storehouse_list = new_store;
}

void Handle_Add_Customer()
{
   int id = Get_Int("Musteri ID giriniz: ");
   printf("Musteri Ismi giriniz: ");
   DString *name = DSTR_GetLine();
```

```
    float x = Get_Float("X Koordinati: ");
    float y = Get_Float("Y Koordinati: ");

    Customer *c = CUS_Create(id, name, x, y);
    if (c == NULL)
    {
        printf("[HATA] : Musteri eklenemedi.\n");
        return;
    }
    Add_To_Customer_List(c);
    printf("Musteri basariyla eklendi!\n");
}

void Handle_Add_Product()
{
    int id = Get_Int("Urun ID giriniz: ");
    printf("Urun Adi giriniz: ");
    DString *name = DSTR_GetLine();
    float price = Get_Float("Birim Fiyat (TL): ");
    int qty = Get_Int("Stok Miktari: ");

    Product *p = PROD_Create(id, name, price, qty);

    if (p == NULL)
    {
        printf("[HATA] : Urun eklenemedi.\n");
        return;
    }

    Add_To_Product_List(p);
    printf("Urun basariyla eklendi!\n");
}

void Handle_Add_Storehouse()
{
    int id = Get_Int("Depo ID giriniz: ");
    float x = Get_Float("Depo X Koordinati: ");
    float y = Get_Float("Depo Y Koordinati: ");

    Storehouse *s = STORE_Create(id, x, y);

    if (s == NULL)
    {
        printf("[HATA] : Depo eklenemedi.\n");
        return;
    }

    Add_To_Store_List(s);
    printf("Depo basariyla eklendi!\n");
```

```
}

void View_All_Customers()
{
   Customer *curr = customer_list;
   printf("\n--- MUSTERI LISTESI ---\n");
   while (curr != NULL)
   {
      Object_Present(curr);
      curr = curr->next;
   }
}

void View_All_Products()
{
   Product *curr = product_list;
   printf("\n--- URUN LISTESI ---\n");
   while (curr != NULL)
   {
      Object_Present(curr);
      curr = curr->next;
   }
}

void View_All_Storehouses()
{
   Storehouse *curr = storehouse_list;
   printf("\n--- DEPO LISTESI ---\n");
   while (curr != NULL)
   {
      Object_Present(curr);
      curr = curr->next;
   }
}

void Analyze_Purchase()
{

   if (customer_list == NULL || product_list == NULL || storehouse_list == NULL)
   {
      printf("[HATA] Analiz yapabilmek icin en az 1 musteri, 1 urun ve 1 depo eklemelisiniz!\n");
      return;
   }

   printf("\n--- SATIN ALMA ISLEMI ---\n");

   View_All_Customers();

   int c_id = Get_Int("Musteri ID giriniz: ");
```

```
Customer *curr_c = customer_list;
while (curr_c != NULL && curr_c->id != c_id)
{
   curr_c = curr_c->next;
}
if (curr_c == NULL)
{
   printf("[HATA] %d ID'li musteri bulunamadi!\n", c_id);
   return;
}

View_All_Products();

int p_id = Get_Int("Satin alinacak Urun ID giriniz: ");

Product *curr_p = product_list;
while (curr_p != NULL && curr_p->id != p_id)
{
   curr_p = curr_p->next;
}
if (curr_p == NULL)
{
   printf("[HATA] %d ID'li urun bulunamadi!\n", p_id);
   return;
}

int buy_qty = Get_Int("Kac adet alinacak?: ");

if (curr_p->quantity < buy_qty)
{
   printf("[HATA] Yetersiz stok! (Mevcut Stok: %d)\n", curr_p->quantity);
   return;
}

Storehouse *curr_s = storehouse_list;
Storehouse *nearest_s = NULL;
float min_dist = -1.0f;

while (curr_s != NULL)
{
   float dist = LOC_CalculateDistance(curr_c->location, curr_s->location);

   if (min_dist < 0 || dist < min_dist)
   {
      min_dist = dist;
      nearest_s = curr_s;
   }
   curr_s = curr_s->next;
```

```
    }

    if (nearest_s == NULL)
    {
        printf("[HATA] Gecerli bir depo hesabi yapilamadi!\n");
        return;
    }

    float product_cost = curr_p->price * buy_qty;
    float shipping_cost = min_dist * 5.0f;
    float total_cost = product_cost + shipping_cost;

    curr_p->quantity -= buy_qty;

    printf("\n================ FATURA VE ANALIZ ================\n");
    printf("Satin Alan Musteri ID : %d\n", curr_c->id);
    printf("Alinan Urun ID       : %d (Miktar: %d adet)\n", curr_p->id, buy_qty);
    printf("Gonderici Depo ID    : %d\n", nearest_s->id);
    printf("Mesafe               : %.2f km\n", min_dist);
    printf("--------------------------------------------------\n");
    printf("Urun Bedeli          : %.2f TL\n", product_cost);
    printf("Kargo Bedeli         : %.2f TL (%.2f x 5 TL)\n", shipping_cost, min_dist);
    printf("TOPLAM TUTAR         : %.2f TL\n", total_cost);
    printf("Kalan Urun Stoku     : %d\n", curr_p->quantity);
    printf("==================================================\n");
}

void Quit_System()
{
    Customer *curr_c = customer_list;
    Customer *next_c;
    while (curr_c != NULL)
    {
        next_c = curr_c->next;
        Object_Kill(curr_c);
        curr_c = next_c;
    }

    Product *curr_p = product_list;
    Product *next_p;
    while (curr_p != NULL)
    {
        next_p = curr_p->next;
        Object_Kill(curr_p);
        curr_p = next_p;
    }

    Storehouse *curr_s = storehouse_list;
    Storehouse *next_s;
```

```
    while (curr_s != NULL)
    {
        next_s = curr_s->next;
        Object_Kill(curr_s);
        curr_s = next_s;
    }

    printf("Sistemden cikiliyor...\n");
}
```

References / Kaynaklar

[1] Deitel & Deitel, “C – How to Program”, Prentice Hall, Seventh Edition

[2] Hanly & Koffman., “Problem Solving and Program Design in C”, Pearson